

Date: _____

Lab # 03 TO UNDERSTAND AND OBSERVE THE RUNNING TIME OF THREE SIMPLE SORTING ALGORITHMS, SELECTION SORT, BUBBLE SORT AND INSERTION SORT

Objective:

- Understand and apply basic sorting algorithms.

Theory:

Sorting algorithms are methods used to arrange a list of items (like numbers) in a specific order, such as from smallest to largest. In this lab, we will explore three simple sorting methods: **Selection Sort**, **Bubble Sort**, and **Insertion Sort**.

1. Selection Sort:

1. This algorithm works by repeatedly finding the smallest (or largest) element from the unsorted portion of the list and swapping it with the first unsorted element.
2. Imagine you have a deck of cards, and you want to sort them by value. You look through the cards to find the smallest card and move it to the front, then repeat this process with the rest of the cards.
3. This takes some time because for every element, you have to compare it with many other elements to find the smallest one.

2. Bubble Sort:

1. Bubble Sort works by repeatedly going through the list and comparing each pair of adjacent elements. If the pair is in the wrong order (the first is bigger than the second), it swaps them.
2. Picture a row of bubbles of different sizes. The bigger bubbles "bubble up" to the top as they swap places with the smaller bubbles. The process continues until all the bubbles (or numbers) are in the right order.
3. This can take a while, especially if the list is long, because it has to go through the entire list multiple times.

3. Insertion Sort:

1. Insertion Sort builds the sorted list one item at a time. It looks at each element and inserts it into its correct position in the already sorted portion of the list.
2. Think about sorting a hand of playing cards. As you pick up each card, you place it in the correct position relative to the cards you're already holding.
3. If the list is almost sorted already, this algorithm can be pretty quick. But if it's all jumbled, it takes more time.

Procedure:

1. **Implementation of Sorting Algorithms:** Start by writing simple Python programs for each sorting algorithm.

a) Selection Sort:

```
def selection_sort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

b) Bubble Sort:

```
def bubble_sort(arr):
    for i in range(len(arr)):
        for j in range(0, len(arr) - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

c) Insertion Sort:

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and key < arr[j]:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
```

2. **Testing the Sorting Algorithms:** Test each sorting algorithm using a small list of numbers (e.g., 10-20 numbers).

Example list:

```
python
```

```
arr = [64, 25, 12, 22, 11]
```

3. **Measuring the Time:** Measure the time it takes to run each algorithm using Python's time module.

```
python
```

```
import time

start_time = time.time()
selection_sort(arr)
end_time = time.time()

print(f"Selection Sort Time: {end_time - start_time} seconds")
```

Exercise

1. Write a program to implement **Bubble Sort**. The program should take an array of integers from the user and sort the elements in **ascending order**. Display the original and sorted arrays.
2. Write a program to implement **Insertion Sort**. The program should take an array of integers as input and sort the elements in **ascending order**. Display the original and sorted arrays.
3. Write a program to implement **Selection Sort**. The program should take an array of integers from the user and sort the elements in **ascending order**. Display the original and sorted arrays.
4. Write a program to implement **Merge Sort** using the divide-and-conquer approach. The program should take an array of integers as input and sort the elements in **ascending order**. Display the original and sorted arrays.
5. What is the main difference between **Bubble Sort and Insertion Sort**?
6. Why is **Merge Sort** considered more efficient than Bubble Sort, Insertion Sort, and Selection Sort for large datasets?
7. How does **Selection Sort** select the element that should be placed at the correct position during each pass?
8. Which of the four sorting algorithms would you choose for:
 - a) A small and nearly sorted array
 - b) A large dataset

Give a brief reason